

A Survey on Enhancement and Efficiency in Retrieval-Augmented Generation

Guiyan He

Texas A&M University

UIN: 136005919

guiyanhe@tamu.edu

Yi-Hsin Chiang

Texas A&M University

UIN: 137001082

yihsin.chiang@tamu.edu

Xiaochuan (Sean) Ge

Texas A&M University

UIN: 937006752

xiaochuan@tamu.edu

Abstract

Retrieval-Augmented Generation (RAG) has emerged as a critical approach for enhancing large language models with external knowledge. While RAG systems are widely adopted, they face significant challenges in reliability, accuracy, and computational efficiency. This survey systematically examines recent advances in RAG optimization through two primary dimensions: Enhancement (quality and robustness) and Efficiency (speed and resource usage). We provide detailed analysis of representative methods including Self-RAG, RE-RAG, CRAG, FLARE, GraphRAG, RAFT, LLMLingua, and Speculative RAG, offering practical insights for both researchers exploring open problems and engineers implementing production systems. Our focused approach addresses gaps in existing literature by emphasizing concrete optimization techniques rather than broad architectural overviews, revealing emerging trends from static retrieval systems toward agentic, self-correcting architectures.

1 Introduction

Retrieval-Augmented Generation (RAG) represents a paradigm shift in how large language models (LLMs) interact with external knowledge. By combining the generative capabilities of LLMs with the precision of information retrieval systems, RAG addresses fundamental limitations of standalone language models, including knowledge staleness, hallucination, and limited domain expertise. The basic RAG architecture operates through a simple yet powerful mechanism: given a user query, a retriever fetches relevant documents from an external knowledge base, which are then concatenated with the original query to augment the prompt for a generative model. This approach enables LLMs to ground their responses in verifiable, up-to-date information without requiring expensive retraining.

Despite widespread adoption across industries—from customer support chatbots to medi-

cal question-answering systems—RAG faces critical bottlenecks that hinder practical deployment. Reliability remains a primary concern when retrieved documents contain noise, contradictory information, or irrelevant content, leading to degraded output quality or even amplified hallucinations. Additionally, computational costs associated with processing lengthy contexts, performing multiple retrieval operations, and running large generative models create significant efficiency challenges, particularly for latency-sensitive applications and resource-constrained deployments.

1.1 Motivation and Scope

The motivation for this survey stems from a fundamental observation: success in RAG deployment depends critically on optimization. As one industry practitioner noted, "Success depends on optimization." While RAG offers compelling theoretical advantages, real-world applications demand systems that achieve both high accuracy and computational efficiency—goals that often exist in tension with one another.

Current literature provides valuable broad surveys of RAG architectures and applications. However, practitioners implementing production systems often find these surveys lack sufficient depth on specific optimization techniques. When should one employ query rewriting versus document reranking? How can prompt compression maintain semantic coherence while achieving 20x reduction? What are the trade-offs between drafter-verifier architectures and single-model approaches? These practical questions often go unanswered in existing surveys.

This survey addresses these gaps by analyzing RAG through two specific, actionable dimensions:

Enhancement focuses on improving quality and robustness through two complementary approaches. First, retriever optimization encompasses techniques for improving the relevance and utility

of retrieved information, including advanced query rewriting methods, hybrid search models combining dense and sparse retrieval, context-aware retrieval strategies, and web search fallback mechanisms. Second, generator optimization covers methods for helping language models better synthesize retrieved context, focusing on improving factual accuracy, handling noisy or contradictory sources, integrating self-reflection mechanisms, and filtering irrelevant information before generation.

Efficiency addresses speed and resource consumption through data-centric and model-centric approaches. Data-centric optimizations organize, structure, or compress input before it reaches the model, including sophisticated prompt compression techniques, efficient indexing strategies, and caching mechanisms. Model-centric approaches modify the inference pipeline or architecture itself, employing techniques such as speculative decoding with drafter-verifier architectures, parallel processing strategies, and dynamic retrieval triggering based on confidence thresholds.

Our target audience spans two critical communities. For researchers, we identify open problems and emerging directions, including the need for standardized efficiency benchmarks, optimal model size combinations in compound architectures, and methods for handling contradictory information. For engineers building production systems, we provide concrete guidance on technique selection, implementation trade-offs, and performance characteristics based on real-world deployments.

1.2 Gap in Current Literature

Recent comprehensive surveys have made valuable contributions to understanding the RAG landscape. "Retrieval And Structuring Augmented Generation with Large Language Models" by Jiang et al. (2025) provides an extensive overview of RAG architectures and structuring techniques. "Retrieval Augmented Generation Evaluation in the Era of Large Language Models" by Gan et al. (2025) focuses on evaluation methodologies and benchmarks across different RAG configurations. These works establish important foundational understanding and taxonomies.

However, these surveys tend toward breadth over depth, covering many techniques at a high level without detailed implementation guidance. They often lack concrete examples of how to imple-

ment specific optimizations such as dynamic retrieval strategies that trigger based on model confidence, budget-aware compression that allocates different compression ratios to prompt components, or multi-agent architectures that decompose complex queries into subtasks. Additionally, most surveys organize content by component (retriever, generator, etc.) rather than by optimization goal (accuracy vs. efficiency), making it challenging to understand trade-offs.

Our survey addresses this gap through three key contributions. First, we focus strictly on optimization techniques rather than general architecture, providing actionable guidance for practitioners. Second, we organize content by optimization objective (enhancement vs. efficiency), making trade-offs explicit and enabling informed decision-making. Third, we include detailed case studies of representative techniques, explaining not just what they do but how they achieve their results and when to apply them.

2 Preliminaries and Foundations

Before examining specific optimization techniques, we establish foundational understanding of the RAG framework, its core components, and evaluation methodologies.

2.1 RAG Architecture Overview

A typical RAG system consists of two primary components working in concert to combine retrieval precision with generation flexibility.

The **Retriever** component is responsible for fetching relevant documents or passages from a large corpus based on the input query. Modern retrievers fall into several categories based on their matching mechanisms. Dense retrieval models such as Dense Passage Retrieval (DPR) and ColBERT use learned neural embeddings to represent both queries and documents in a shared semantic space, enabling similarity-based matching that captures semantic relatedness beyond keyword overlap. Sparse retrieval models such as BM25 rely on traditional information retrieval techniques using term-based matching, offering computational efficiency and interpretability. Hybrid approaches that combine both paradigms have shown promising results by leveraging complementary strengths: sparse methods excel at exact keyword matching while dense methods capture semantic similarity.

The **Generator** is typically a large language

model that takes retrieved documents as additional context and produces coherent, contextually relevant responses. Popular generator architectures include GPT variants (GPT-3, GPT-4), T5 (Text-to-Text Transfer Transformer), and BART (Bidirectional and Auto-Regressive Transformers). The generator must effectively synthesize information from multiple sources while maintaining factual accuracy, coherence, and faithfulness to the retrieved evidence. This synthesis challenge becomes particularly acute when retrieved documents contain contradictory information or when only portions of retrieved documents are relevant to the query.

2.2 Standard RAG Pipeline

The conventional RAG pipeline operates through a sequential process. Given a user query q , the retriever searches a document corpus $\mathcal{D}$ and returns the top- k most relevant passages $\{d_1, d_2, \dots, d_k\}$ based on a relevance scoring function. These passages are then concatenated with the original query to form an augmented prompt: $p = [q; d_1; d_2; \dots; d_k]$. This augmented prompt is processed by the generator LLM to produce the final response r .

This simple architecture, while effective for many applications, suffers from several inherent limitations that motivate the optimization techniques surveyed in this paper. First, the retriever operates independently of the generator, potentially fetching documents that are semantically similar to the query but not actually useful for generation. Second, all retrieved documents are treated equally regardless of their actual utility, leading to context window bloat and increased inference costs. Third, the system retrieves once upfront rather than dynamically adjusting retrieval based on generation needs. Fourth, there is no mechanism for the system to critique or verify its own outputs, leading to propagation of errors when retrieval is poor.

2.3 Datasets and Evaluation Metrics

Common datasets used for RAG evaluation span different domains and task types. Natural Questions provides open-domain question answering pairs derived from Google search queries with answers from Wikipedia, testing systems' ability to locate and synthesize information from large document collections. TriviaQA focuses on trivia questions requiring factual knowledge, with evidence passages from Wikipedia and web documents. HotpotQA requires multi-hop reasoning

where answering questions necessitates combining information from multiple documents. PubMedQA tests biomedical question answering using research paper abstracts, evaluating domain-specific performance. WebGPT provides web-based question answering with human-written answers and citations, enabling evaluation of both answer quality and attribution.

Standard evaluation metrics assess different aspects of RAG performance. Exact Match (EM) measures the percentage of generated answers that match gold standard answers exactly, providing a strict accuracy measure. F1 Score computes token-level overlap between generated and reference answers, offering more lenient evaluation that credits partial matches. BLEU and ROUGE scores evaluate text similarity at different granularities. Beyond these automatic metrics, human evaluation criteria increasingly assess factual correctness (whether generated information is true), relevance (whether the answer addresses the question), coherence (whether the response is well-structured), and attribution quality (whether claims are properly grounded in retrieved documents).

However, a critical gap exists in evaluation methodologies. Most current benchmarks focus exclusively on accuracy metrics while ignoring efficiency considerations such as latency, throughput, and computational cost. This creates challenges for production deployments where both quality and efficiency matter. Moreover, evaluation often occurs on synthetic datasets rather than real user queries, potentially missing important distribution shifts and edge cases that arise in practice.

3 Enhancement Methods

Enhancement methods aim to improve the accuracy, reliability, and robustness of RAG systems. These techniques address fundamental challenges in ensuring that retrieval identifies truly relevant information and that generation produces factually accurate, well-grounded responses. We organize these methods by whether they primarily optimize the retrieval phase, the generation phase, or both through integrated approaches.

3.1 Retriever Optimization

Retriever optimization focuses on improving the quality, relevance, and utility of retrieved documents before they reach the generator. These techniques recognize that downstream generation qual-

ity depends critically on upstream retrieval performance.

3.1.1 Self-RAG: Learning to Retrieve, Generate, and Critique

Asai et al. (2024, ICLR 2024) identified a fundamental problem with standard RAG architectures: systems retrieve indiscriminately for every query and passively trust whatever context is retrieved, regardless of its quality or relevance. When retrieved documents are irrelevant, contradictory, or of low quality, the model becomes prone to hallucinations or generates misleading answers. Moreover, not every query requires external retrieval—some questions can be answered adequately from the model’s parametric knowledge alone.

Self-RAG introduces a groundbreaking approach where the generator learns to output special "reflection tokens" alongside generated text, enabling the model to actively critique its own retrieval and generation process. This self-reflection capability operates through three complementary mechanisms that transform the model from a passive consumer of retrieved information into an active, critical reasoner.

The first mechanism, retrieval decision via [Retrieve] tokens, allows the model to determine when external information is actually needed for a given query. Rather than retrieving for every input, the model learns to distinguish between queries that can be answered from parametric knowledge (e.g., "What is the capital of France?") and those requiring external information (e.g., "What were Tesla’s Q3 2024 earnings?"). This selective retrieval reduces unnecessary computational overhead and API costs while preventing irrelevant information from degrading generation quality.

The second mechanism, relevance assessment through [IsREL] tokens, enables the model to evaluate whether retrieved documents are genuinely relevant to the query at hand. The model learns to identify when retrieved documents, despite being semantically similar to the query, do not actually contain information useful for answering it. This addresses a critical limitation of traditional similarity-based retrieval: semantic similarity does not guarantee utility. By generating low relevance scores for unhelpful documents, the model can effectively ignore them during generation, preventing distraction and confusion.

The third mechanism, factuality verification via [IsSUP] tokens, allows the model to check whether

its generated answer is properly supported by the retrieved evidence. After generating a response, the model assesses whether each claim in its answer can be traced back to specific statements in the retrieved documents. This introspective capability reduces hallucinations by catching unsupported or speculative statements before they reach the user.

Importantly, these reflection tokens are trained through supervised learning on labeled data where human annotators or automated systems have marked when retrieval is needed, which documents are relevant, and which generated statements are supported. During inference, the model generates these tokens alongside text, using them to make real-time decisions about retrieval, filtering, and generation quality.

The results demonstrate significant improvements over standard RAG across multiple benchmarks. On open-domain question answering tasks, Self-RAG achieves 5-10% absolute improvement in accuracy by reducing errors from irrelevant retrieval and catching hallucinations through factuality checking. The model also exhibits improved robustness: when retrieval quality degrades (e.g., due to corpus noise or adversarial document injection), Self-RAG maintains higher accuracy than baseline RAG by learning to ignore poor-quality retrieved content.

The key insight from Self-RAG extends beyond its specific architecture to a fundamental reconceptualization of RAG. Rather than treating information retrieval metrics (precision, recall, relevance) as external evaluation criteria calculated offline, Self-RAG internalizes these concepts as learned behaviors that the model executes in real-time during inference. This convergence of traditional IR principles with modern LLM capabilities points toward future systems that seamlessly integrate retrieval logic with language understanding and generation.

3.1.2 CRAG: Corrective RAG with Web Search Fallback

CRAG (Yan et al., 2024) addresses a critical failure mode in RAG systems: what happens when initial retrieval from the internal knowledge base fails to find adequate information? Traditional RAG systems have no fallback mechanism—they simply generate based on whatever documents were retrieved, even if those documents are incorrect or insufficient.

CRAG introduces a corrective approach with three key components. First, a relevance evaluator

assesses the quality of initially retrieved documents using both embedding similarity and a learned relevance classifier. Second, if documents are deemed incorrect or confidence is low, the system triggers an external web search (e.g., via Google Search API) to find additional sources. Third, documents from both internal and external sources are combined and reranked before generation, ensuring the most reliable information guides the response.

This corrective mechanism proves particularly valuable in domains where internal knowledge bases may have gaps or where information changes rapidly. In experiments on medical question answering and current events datasets, CRAG shows substantial improvements in answer accuracy (10-15% on time-sensitive queries) by successfully recovering from initial retrieval failures. The web search fallback also improves system robustness: even when internal retrievers are adversarially poisoned or degraded, CRAG maintains reasonable performance by accessing external sources.

The practical implications are significant for production deployments. By implementing a fallback strategy, systems become more resilient to knowledge base incompleteness—a common real-world challenge where internal corpora inevitably contain gaps. However, this approach introduces additional considerations: web search adds latency and API costs, external results may require additional verification, and careful filtering is needed to prevent misinformation from unreliable web sources from degrading answer quality.

3.1.3 FLARE: Forward-Looking Active Retrieval

FLARE (Jiang et al., 2023) challenges the standard RAG assumption that all retrieval should occur upfront before generation begins. Instead, FLARE introduces active, dynamic retrieval that pauses generation when the model’s confidence drops, retrieves additional facts, and then resumes generation with augmented context.

The system operates through continuous confidence monitoring during generation. As the model produces text, FLARE calculates token-level or sentence-level confidence scores based on probability distributions from the language model. When confidence falls below a learned threshold—indicating uncertainty or knowledge gaps—the system pauses generation, formulates a search query based on the partial response and query context, retrieves additional relevant docu-

ments, and incorporates them into the context before continuing generation.

This dynamic approach offers several advantages over static upfront retrieval. First, it retrieves only when necessary, reducing computational overhead for queries where the model’s initial context is sufficient. Second, it can retrieve multiple times during generation for complex queries requiring information from different sources at different stages. Third, confidence-based triggering ensures retrieval occurs precisely when the model needs additional information, rather than following a fixed retrieval schedule.

FLARE demonstrates particular effectiveness on long-form generation tasks such as biography writing and report generation, where different sections may require different sources of information. By retrieving incrementally as generation progresses, FLARE maintains higher factual accuracy throughout extended responses compared to systems that retrieve all context upfront and then generate without further information access.

However, dynamic retrieval introduces implementation complexity. Multiple retrieval operations increase total latency, though strategic batching and caching can mitigate this. Confidence estimation requires careful calibration—thresholds that are too low result in over-retrieval and wasted computation, while thresholds that are too high miss opportunities to prevent errors. Despite these challenges, FLARE’s paradigm of confidence-aware, dynamic retrieval represents an important direction for next-generation RAG systems that adaptively allocate computational resources based on query difficulty and model certainty.

3.2 Generator Optimization

Generator optimization techniques focus on improving how language models process and synthesize retrieved information, addressing challenges such as handling noisy or contradictory documents, maintaining factual accuracy, and efficiently utilizing limited context windows.

3.2.1 RE-RAG: Relevance Estimator for Filtering Noisy Context

Kim et al. (2024, EMNLP 2024) tackled a counter-intuitive problem in RAG: providing more context does not always improve generation quality. When retrieved documents contain noise, irrelevant information, or contradictory claims, LLMs can become confused or distracted, leading to degraded accu-

racy compared to retrieval systems that provide less but higher-quality context.

RE-RAG introduces an external Relevance Estimator (RE) component that sits between retrieval and generation, filtering contexts before they reach the LLM. Unlike traditional retrieval scoring that focuses on semantic similarity between query and document embeddings, the relevance estimator assesses utility: does this document actually help answer this specific question? This distinction proves crucial. A document might be highly similar to the query semantically (high cosine similarity in embedding space) but still provide no useful information for answering it. Conversely, a document with moderate semantic similarity might contain the exact fact needed.

The relevance estimator is typically implemented as a trained classifier or scoring model that takes as input the query, the retrieved document, and optionally the candidate answer, and outputs a relevance score. Training data can be constructed through several approaches: human annotation of query-document-answer triples for relevance, automatic labeling based on whether using a document improves answer quality, or reinforcement learning where the estimator learns from downstream generation success.

During inference, the relevance estimator scores all retrieved documents and either reranks them (keeping top-k by utility rather than similarity) or applies hard filtering (discarding documents below a relevance threshold). This filtered, reranked set of documents then augments the prompt for generation.

Experimental results demonstrate the power of utility-based filtering. On multi-document question answering datasets such as Natural Questions and TriviaQA, RE-RAG improves accuracy by 8-12% compared to standard RAG that uses all retrieved documents. Importantly, RE-RAG often achieves better performance while using fewer documents, demonstrating that quality trumps quantity for context augmentation.

The approach also significantly improves interpretability and debuggability in production systems. By making explicit which documents contributed to an answer (those passing the relevance filter) and which were deemed unhelpful, developers can better understand system behavior, diagnose failures, and refine retrieval strategies. This transparency proves valuable for user trust in high-stakes do-

main tasks such as medical diagnosis or legal research.

However, RE-RAG introduces an additional component that requires training and inference computation. The relevance estimator adds latency to the pipeline and necessitates domain-specific training data. For applications where retrieval quality is already high and documents are uniformly relevant, the added complexity may not justify the benefits. Conversely, in noisy environments or when using multiple heterogeneous retrieval sources, relevance estimation becomes increasingly valuable.

3.3 Integrated Approaches: Combining Retrieval and Generation Optimization

Several recent techniques optimize both retrieval and generation jointly, recognizing that these components interact in complex ways that affect overall system performance.

3.3.1 GraphRAG: Knowledge Graph-Based Retrieval

Microsoft Research's GraphRAG (Edge et al., 2024) represents a fundamental rethinking of how RAG systems represent and retrieve knowledge. While traditional RAG treats the document corpus as a flat collection of text chunks indexed by embeddings, GraphRAG leverages LLMs to construct a knowledge graph from the corpus, then uses graph-based retrieval and reasoning to answer queries.

The GraphRAG pipeline consists of several stages. First, during indexing, an LLM reads through source documents and extracts entities (people, places, organizations, concepts) and relationships between them (e.g., "Einstein developed the theory of relativity"). These extracted elements form nodes and edges in a knowledge graph. Second, community detection algorithms (such as the Leiden algorithm) identify clusters of highly connected entities, representing coherent subtopics within the corpus. Third, the system generates hierarchical summaries for each community at multiple levels of granularity, from fine-grained entity descriptions to high-level thematic overviews.

During query processing, GraphRAG operates differently for local versus global queries. For local queries that ask about specific entities (e.g., "What is Novorossiysk's relationship to the 2008 Moscow bombings?"), the system identifies relevant entities in the query, retrieves their local graph neighborhoods, and uses this structured context to generate answers. For global queries that ask about themes across the

entire corpus (e.g., "What are the main themes in the dataset?"), traditional RAG fails because simple similarity search cannot identify a small set of chunks that comprehensively represent corpus-wide patterns. GraphRAG addresses this through a map-reduce approach: it generates answers from each community summary (map step), then synthesizes these community-level answers into a final global response (reduce step).

Empirical evaluations demonstrate GraphRAG's substantial advantages on certain query types. On global sensemaking questions about news articles, GraphRAG outperforms naive RAG by 100-130% in answer comprehensiveness and by 50-70% in diversity, measured through human evaluation. For multi-hop reasoning queries requiring connection of disparate pieces of information, GraphRAG's graph structure enables traversal of relationship chains that flat vector search cannot easily discover.

However, GraphRAG introduces significant upfront costs. Constructing the knowledge graph requires multiple LLM inference passes over the entire corpus for entity extraction, relationship identification, and summarization—potentially 10-50x more expensive than simple embedding-based indexing. These costs must be amortized over many queries to justify the investment. Additionally, GraphRAG's advantages are most pronounced for complex analytical queries; for simple factoid questions, traditional RAG often performs comparably at much lower cost.

Microsoft's development of prompt auto-tuning for GraphRAG addresses one major challenge: domain adaptation. Different corpora require different entity types and relationship schemas. For chemistry documents, molecules and reactions are key entities; for organizational documents, people and roles matter most. GraphRAG's auto-tuning approach uses the LLM to analyze a sample of documents and automatically suggest appropriate entity types, relationship types, and extraction prompts, significantly reducing the manual effort required to customize GraphRAG for new domains.

3.3.2 RAFT: Retrieval-Augmented Fine-Tuning

Zhang et al. (2024) introduced Retrieval-Augmented Fine-Tuning (RAFT), which bridges the gap between two traditionally separate approaches for domain adaptation: supervised fine-tuning (SFT) and RAG. The key insight is that a model explicitly trained to extract information from

retrieved documents in an "open-book exam" setting outperforms both models that rely solely on memorized knowledge and models that use RAG without prior training on how to effectively leverage retrieved context.

RAFT's training methodology is distinctive. The system first creates a synthetic training dataset where each example consists of a question, a set of documents including both "oracle" documents that contain the answer and "distractor" documents that do not, and a chain-of-thought-style answer that cites specific passages from the oracle documents. During training, the model receives the question and all documents (both oracle and distractors) and must learn to identify which documents are helpful, extract relevant information, and generate well-grounded answers.

Critically, RAFT includes a stochastic component: in a fraction P of training examples, the oracle document is removed, providing only distractors. This forces the model to learn what to do when no retrieved document contains the answer—either declining to answer or acknowledging uncertainty—rather than hallucinating information or misinterpreting distractor documents as relevant.

The chain-of-thought reasoning component proves essential for performance. Rather than simply training the model to map questions plus documents to answers, RAFT teaches the model to explain its reasoning: "According to document D2, [relevant quote], which indicates that [interpretation], therefore the answer is [conclusion]." This reasoning process improves both accuracy and interpretability while reducing overfitting during training.

Experimental results across multiple domains demonstrate RAFT's effectiveness. On PubMed biomedical question answering, RAFT fine-tuned models achieve 7-9% higher accuracy than base models with RAG. On HotpotQA multi-hop reasoning, improvements reach 30-35%. Compared to domain-specific fine-tuning without RAG, RAFT consistently outperforms by 15-20% because it learns both domain knowledge and how to effectively use retrieved context.

RAFT occupies a valuable middle ground in the fine-tuning versus RAG trade-off space. Pure RAG requires no training but may not optimally leverage retrieved documents, especially in specialized domains. Pure domain-specific fine-tuning bakes knowledge into parameters but cannot adapt to new

information or cite sources. RAFT combines advantages: domain-specific training that teaches effective context utilization while maintaining the flexibility to incorporate new documents at inference time.

From a practical perspective, RAFT is particularly well-suited for scenarios with a stable domain-specific document corpus (e.g., company documentation, medical guidelines, legal precedents) where the benefits of specialized training justify the upfront investment. The approach proves less suitable for rapidly changing information or extremely broad domains where maintaining representative training data becomes infeasible.

4 Efficiency Methods

As RAG systems scale to enterprise workloads, the research focus in 2024–2025 has pivoted from pure generation quality to addressing critical bottlenecks in latency, memory bandwidth, and inference cost. The expansion of context windows to accommodate extensive retrieval sets has introduced quadratic computational complexity and substantial Time-To-First-Token (TTFT) delays. Recent optimizations can be categorized into Data-Centric methods, which refine the input payload, and Model-Centric methods, which restructure the inference process.

4.1 Data-Centric Optimizations

Data-centric methods aim to maximize the information density of the input prompt. By compressing contexts or caching intermediate states, these approaches reduce the arithmetic intensity of processing retrieved documents without altering the core model architecture.

4.1.1 Prompt Compression and Context Filtering

Processing long, concatenated documents often leads to the "lost-in-the-middle" phenomenon and high API costs. Research has moved beyond heuristic truncation toward information-theoretic and generative compression.

Information-Theoretic Pruning: LongLLMLingua (Jiang et al., 2024a) reconceptualizes the prompt as an information channel. It employs a coarse-to-fine compression strategy with a Budget Controller that dynamically allocates token quotas across instructions and documents. By utilizing Contrastive Perplexity—measuring the conditional information gain of a token relative to the query—and a Subsequence Recovery mechanism

to preserve syntactic structure, it achieves up to 20x compression while enhancing performance on benchmarks like NaturalQuestions by mitigating noise.

Generative Context Synthesis: Unlike extractive methods that risk structural incoherence, SCOPE (Zhang et al., 2025) introduces a generative "chunking-and-summarization" mechanism. It splits context into semantic chunks, identifies outliers in vector space, and uses a summarizer to rewrite content into concise, syntactically complete statements. SCOPE employs a Dynamic Compression Ratio to maintain narrative flow, demonstrating superior stability at high compression rates compared to token-dropping methods.

Attention-Guided Pruning: AttentionRAG (Fang et al., 2025) leverages the LLM's own attention mechanism to guide pruning. It reformulates the query into a "next-token prediction" task to identify a "focal token" (e.g., the semantic center of the query). The system retains only those context tokens that receive high attention scores from this focal token. This Attention Focus Mechanism achieves 6.3x compression and outperforms generic perplexity-based methods by 10% in accuracy by precisely isolating query-relevant evidence.

4.1.2 Knowledge State Management and KV Caching

A dominant trend in 2025 is the shift from caching raw text to caching Key-Value (KV) model states. Since retrieved documents are often static across queries, recomputing their KV tensors represents redundant computation.

Prefix Caching Structures: RAGCache (Jin et al., 2025) organizes cached states into a "Knowledge Tree" where common instruction prefixes form the root and document sequences form branches. It utilizes a PGDSF (Prefix-aware Greedy-Dual-Size-Frequency) replacement policy to manage GPU memory. By retrieving pre-computed tensors for the longest matching prefix, RAGCache reduces TTFT by 4x compared to standard serving systems like vLLM.

Non-Prefix State Fusion: Standard caching fails when documents appear in the middle of a prompt due to positional dependency. CacheBlend (Yao et al., 2025) solves this via selective KV recomputation. It identifies "Highly KVD" (Key-Value Deviation) tokens—those most affected by positional shifts—and recomputes only this small subset (top- k deviation), blending them with cached

states. This achieves 100% KV cache hit rates for non-prefix chunks, improving throughput by 2.8–5x.

Offline-Online Hybrid: TurboRAG (Lu et al., 2025) moves the encoding of the entire document corpus to an offline phase. During inference, it retrieves pre-computed KV tensors rather than text. Using "Independent-Attention" and "Reordered-RoPE" mechanisms to stitch disjoint caches dynamically, TurboRAG effectively eliminates online prefill latency, reducing TTFT by up to 9.4x.

Reranker Optimization: HyperRAG (An et al., 2025) extends caching to the reranker. By caching document-side KV states, it allows the query to cross-attend to pre-encoded documents without full re-processing. This improves reranking throughput by 2–3x, enabling the use of powerful decoder-only rerankers (e.g., Gemma-2B) at the speed of lighter models.

4.1.3 Efficient Indexing and Knowledge Representation

Optimizing the structure of the retrieval target itself is crucial for reducing search latency and memory footprint.

Hardware-Aware Partitioning: VectorLiteRAG (Kim and Mahajan, 2025) addresses GPU memory contention between the vector index and the LLM’s KV cache. Exploiting the Zipfian distribution of cluster access, it dynamically partitions the index, pinning "hot" clusters to GPU HBM and relegating "cold" ones to CPU DRAM. This fine-grained strategy improves SLO-compliant throughput by 1.5x without additional hardware.

Structured Hypergraphs: HyperGraphRAG (Luo et al., 2025) advances beyond binary graphs by using hyperedges to model n-ary relations ($n \geq 2$). This structure retrieves complex dependencies (e.g., multi-entity interactions) in a single hop, reducing the token overhead of expressing complex facts and minimizing iterative retrieval steps.

4.2 Model-Centric Optimizations

Model-centric approaches modify the inference behavior of the LLM, employing speculation, parallelization, and adaptive routing to process retrieved contexts more efficiently.

4.2.1 Speculative and Parallel Decoding

Adapting speculative decoding (SD) to RAG allows systems to bypass the arithmetic intensity of autoregressive generation for grounded content.

Specialist Drafting: Speculative RAG (Wang et al., 2024) replaces standard draft models with a specialist RAG drafter. This smaller model generates multiple drafts in parallel from distinct subsets of retrieved documents, creating a "multi-perspective" draft set. A larger generalist model verifies these drafts in a single pass. This method enhances accuracy by 12.97% while reducing latency by over 50%, as the specialist effectively offloads the reasoning burden.

Retrieval-Based Drafting: REST (He et al., 2024) utilizes a non-parametric approach, retrieving high-frequency n-grams or phrases from a datastore to serve as draft tokens. This training-free method converts generation into a retrieval-verification loop, achieving speedups of 1.62x–2.36x by skipping generation for common phrases.

Long-Context Transfer: RAPID (Chen et al.) targets long-context RAG. It employs a "RAG Drafter" that processes a compressed context to speculate on the output of a target model processing the full context. An inference-time knowledge transfer aligns the distributions, achieving >2x speedups for memory-bound long-context tasks.

4.2.2 Architecture and Attention Optimization

Linearizing Attention: Block-Attention (Ma et al., 2024) modifies the attention mask to linearize prefill complexity. It restricts tokens in a retrieved chunk to attend only to local tokens (within the same chunk) and the query, preventing expensive cross-document attention. This reduces FLOPs and TTFT by over 98

4.2.3 Adaptive and Dynamic Execution

Complexity-Aware Routing: Adaptive-RAG (Jeong et al., 2024) employs a complexity classifier to dynamically route queries. "Simple" queries bypass retrieval entirely (using parametric memory), while "Complex" queries trigger multi-step retrieval. This prevents resource waste on trivial tasks.

Agentic Feedback: FAIR-RAG (Asl et al., 2025) integrates a "Structured Evidence Assessment" (SEA) module. This agentic loop critiques retrieved evidence for gaps before generation, triggering targeted sub-queries only when necessary, thereby optimizing the computation-per-correct-answer ratio.

4.2.4 System-Algorithm Co-Design

Pipeline Parallelism: PipeRAG (Jiang et al., 2024b) introduces pipeline parallelism to overlap retrieval and generation. By dynamically determining a "flexible retrieval interval" based on a performance model, it allows the GPU to process available chunks while the retriever fetches subsequent ones, effectively hiding retrieval latency and achieving a 2.6x speedup.

Joint Scheduling: METIS (Ray et al., 2025) is the first system to jointly optimize query scheduling and RAG configuration. It profiles query complexity to tailor the number of retrieved chunks and assigns requests to GPUs to minimize queue contention. This co-optimization reduces generation latency by 1.64x–2.54x.

5 Conclusion

This survey has examined optimization techniques across two primary dimensions: enhancement (quality and robustness) and efficiency (speed and resource consumption). The methods surveyed—from Self-RAG’s introspective retrieval and generation critique to LLMLingua’s semantic-aware compression—represent significant advances in making RAG systems both more accurate and more practical for production deployment. Future directions include developing standardized benchmarks that jointly evaluate accuracy and efficiency trade-offs, exploring optimal combinations of multiple optimization techniques, and extending these approaches to multimodal and multilingual RAG systems.

References

- Yuwei An, Yihua Cheng, Seo Jin Park, and Junchen Jiang. 2025. Hyperrag: Enhancing quality-efficiency tradeoffs in retrieval-augmented generation with reranker kv-cache reuse. *arXiv preprint arXiv:2504.02921*.
- Mohammad Aghajani Asl, Majid Asgari-Bidhendi, and Behrooz Minaei-Bidgoli. 2025. Fair-rag: Faithful adaptive iterative refinement for retrieval-augmented generation. *arXiv preprint arXiv:2510.22344*.
- Guanzheng Chen, Qilong Feng, Jinjie Ni, Xin Li, and Michael Qizhe Shieh. Rapid: Long-context inference with retrieval-augmented speculative decoding. In *Forty-second International Conference on Machine Learning*.
- Yixiong Fang, Tianran Sun, Yuling Shi, and Xiaodong Gu. 2025. Attentionrag: Attention-guided context pruning in retrieval-augmented generation. *arXiv preprint arXiv:2503.10720*.
- Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 conference of the North American chapter of the association for computational linguistics: Human language technologies (volume 1: long papers)*, pages 1582–1595.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403*.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024a. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677.
- Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. 2024b. Piperag: Fast retrieval-augmented generation via algorithm-system co-design. *arXiv preprint arXiv:2403.05676*.
- Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. Raggcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):1–27.
- Junkyum Kim and Divya Mahajan. 2025. An adaptive vector index partitioning scheme for low-latency rag pipeline. *arXiv preprint arXiv:2504.08930*.
- Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2025. Turborag: Accelerating retrieval-augmented generation with precomputed kv caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6599–6612.
- Haoran Luo, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, et al. 2025. Hypergraphrag: Retrieval-augmented generation via hypergraph-structured knowledge representation. *arXiv preprint arXiv:2503.21322*.
- Dongyang Ma, Yan Wang, and Lan Tian. 2024. Block-attention for efficient prefilling. *arXiv preprint arXiv:2409.15355*.
- Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. Metis: Fast quality-aware rag systems with configuration adaptation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 606–622.

Zilong Wang, Zifeng Wang, Long Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, et al. 2024. Speculative rag: Enhancing retrieval augmented generation through drafting. *arXiv preprint arXiv:2407.08223*.

Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 94–109.

Tinghui Zhang, Yifan Wang, and Daisy Zhe Wang. 2025. Scope: A generative approach for llm prompt compression. *arXiv preprint arXiv:2508.15813*.